

Безопасная обработка .env-файлов

Акинжала Родион, Ибрагимова Ксения, Гудков Денис

1 Формальная постановка задачи

Файлы формата `.env` используются для задания переменных окружения. Обычно строка такого файла имеет вид `KEY=value`. При этом формат может поддерживать подстановку значений других переменных, например:

```
NAME=Ivan
GREETING="Hello, $NAME"
```

В этом примере значение переменной `GREETING` должно быть вычислено как строка `Hello, Ivan`.

Нельзя безопасно обрабатывать `.env`-файл как shell-скрипт, например командой `source file.env`, потому что внутри файла могут находиться конструкции для исполнения команд:

```
PASSWORD=$(cat /etc/passwd)
X='whoami'
```

Требуется реализовать программу, которая безопасно читает `.env`-файл, разбирает его с помощью синтаксического анализа, вычисляет значения переменных с учётом подстановок и выводит результат в JSON. При этом содержимое файла не должно выполняться как shell-код.

В работе поддерживаются:

- присваивания вида `NAME=value`;
- необязательный префикс `export`;
- значения без кавычек;
- значения в одинарных и двойных кавычках;
- комментарии;
- подстановки переменных вида `$VAR` и `${VAR}`.

Запрещаются:

- конструкции вида `$(...)`;

- неэкранированные бэктики;
- неизвестные переменные;
- повторные определения переменных;
- циклические зависимости между переменными.

2 Грамматика

Формат `.env` описывается грамматикой в нотации PEG. В грамматике используется упорядоченный выбор: если первый вариант не подходит, проверяется следующий.

2.1 Файл и строки

```
EnvFile = Line*
Line    = Blanks (Assignment / Comment / "")
Blanks  = (" " / "\t")*
```

Файл состоит из строк. Каждая строка является присваиванием, комментарием или пустой строкой. Ведущие пробелы и табы игнорируются.

2.2 Присваивание

```
Assignment = ("export" " " "+)? Name Blanks "=" Blanks Value
Name       = [a-zA-Z_] [a-zA-Z0-9_]*
```

Имя переменной начинается с буквы или символа подчёркивания. Далее могут идти буквы, цифры и символ подчёркивания. Пробелы вокруг знака `=` допустимы.

2.3 Значение

```
Value = DoubleQuoted / SingleQuoted / Raw
```

Поддерживаются три вида значений: в двойных кавычках, в одинарных кавычках и без кавычек.

2.4 Двойные кавычки

```
DoubleQuoted = '"' DblChunk* '"' Blanks Comment?
DblChunk      = Escape / VarRef / (!'"' !$' !'\\" !'\' ANY)
```

Внутри двойных кавычек работают подстановки переменных и escape-последовательности. Обычный символ не может быть кавычкой, знаком доллара, обратным слэшем или бэк-тиком.

После закрывающей кавычки в той же строке допустимы только пробелы, табуляции и необязательный комментарий. Любые другие символы после закрывающей кавычки приводят к ошибке разбора.

2.5 Одинарные кавычки

```
SingleQuoted = "'" SqContent* "'"
SqContent    = !'\' !"$(" !'" ANY
```

Внутри одинарных кавычек содержимое считается буквальным текстом. Подстановки переменных там не выполняются.

2.6 Значение без кавычек

```
Raw          = RawChunk* Blanks Comment?
RawChunk     = VarRef / (!'\' !'#" !'\' !"$(" ANY)
```

Значение без кавычек читается до конца строки или до символа комментария #. В таких значениях работают подстановки переменных.

Пробелы и табуляции в конце значения, то есть между последним значащим символом и символом # или концом строки, отбрасываются и в итоговое значение не попадают. Например, в строке `A=hello # sm` итоговым значением переменной `A` будет `hello`, а не `hello .`

2.7 Подстановка переменных

```
VarRef = "${" Name "}" / "$" Name
```

Подстановка работает в двойных кавычках и в значениях без кавычек. В одинарных кавычках подстановка не выполняется.

2.8 Escape-последовательности и комментарии

```
Escape = "\\\" EscChar
EscChar = "n" / "t" / "r" / "\\\" / '\\" / "$" / "'
```

```
Comment = "#" ANY*
```

Escape-последовательности используются только внутри двойных кавычек. Комментарий начинается с символа # и идёт до конца строки.

3 Внутреннее представление

Результатом синтаксического анализа является дерево, описывающее переменные и их значения. На верхнем уровне хранится список определений переменных:

```
EnvFile
  Binding Name Value
```

Значение переменной может иметь один из трёх видов:

```
Raw [Chunk]
SingleQuoted String
DoubleQuoted [Chunk]
```

Фрагмент значения имеет один из двух типов:

```
TextChunk String
VarRef Name
```

Обычный текст хранится как `TextChunk`. Ссылка на другую переменную хранится как `VarRef`. Благодаря этому парсер не вычисляет значения сразу, а только строит структуру строки.

Например строка

```
GREETING="Hello, $NAME"
```

преобразуется в значение вида:

```
DoubleQuoted
  [ TextChunk "Hello, "
    , VarRef "NAME"
  ]
```

4 Алгоритмы

4.1 Синтаксический анализ

Парсер реализует PEG-грамматику методом рекурсивных функций. Каждая функция соответствует отдельному правилу грамматики: например, `parseEnvFile` разбирает файл, `parseAssignment` разбирает присваивание, `parseDoubleChunks` разбирает содержимое двойных кавычек.

Алгоритм работы парсера:

1. входной текст разбивается на строки;
2. каждая строка разбирается как пустая строка, комментарий или присваивание;
3. в присваивании выделяются имя переменной и значение;
4. значение разбирается как `DoubleQuoted`, `SingleQuoted` или `Raw`;

5. внутри значения выделяются текстовые фрагменты и ссылки на переменные;
6. опасные конструкции `$(...)` и незранированные бэктики приводят к ошибке.

После того как список фрагментов получен, выполняется нормализация: соседние фрагменты вида `TextChunk` склеиваются в один, а пустые фрагменты удаляются. Это упрощает дальнейшие проходы по дереву и делает представление однозначным.

Отдельно оговаривается случай, когда знак доллара встречается не в составе ссылки на переменную. По грамматике после `$` должно идти имя переменной либо в виде `$NAME`, либо в виде `${NAME}`. Если после `$` нет ни одной буквы или подчёркивания, парсер не считает это ошибкой, а оставляет знак доллара как часть обычного текста. Например, в значении `"price $5"` символ `$` сохраняется буквально и попадает в итоговую строку.

Таким образом, файл не выполняется как `shell`-код, а только разбирается как текст по заданной грамматике.

4.2 Вычисление значений

После построения синтаксического дерева выполняется семантический проход. Он преобразует разобранные значения в итоговые строки.

Вычисление выполняется в два этапа:

1. функция `buildDefs` строит таблицу определений вида `[(имя, AST-значение)]` и проверяет, что одна переменная не определена дважды;
2. функция `evalVar` рекурсивно вычисляет значение переменной, подставляя значения других переменных вместо `VarRef`.

Если фрагмент имеет вид `TextChunk`, он добавляется к результату без изменений. Если фрагмент имеет вид `VarRef`, вычислитель ищет соответствующую переменную, вычисляет её значение и подставляет его в текущую строку.

Например для файла

```
NAME=Ivan
GREETING="Hello, $NAME"
```

переменная `GREETING` содержит два фрагмента: обычный текст `Hello,` и ссылку на переменную `NAME`. После вычисления ссылка заменяется значением `Ivan`, и итоговая строка становится `Hello, Ivan`.

4.3 Обнаружение циклов

Переменные могут ссылаться друг на друга. Чтобы программа не заикливалась, при вычислении хранится текущий путь зависимостей.

Например:

```
A=$B  
B=$C  
C=$A
```

При вычислении переменной **A** программа переходит к **B**, затем к **C**, а затем снова к **A**. Так обнаруживается цикл, и вычисление завершается ошибкой.

4.4 Вывод результата

После вычисления всех переменных получается список пар:

```
[(имя, значение)]
```

Модуль вывода преобразует этот список в JSON-объект. Специальные символы, такие как кавычки, обратные слэши, переносы строк и табуляции, экранируются, чтобы результат оставался корректным JSON.